APÊNDICE TÉCNICO: ARQUITETURA DE SOFTWARE E DEMONSTRAÇÃO DE CÓDIGOS

Engenharia de Software, Schemas Relacionais, Pipelines CRISP-DM e Implementação dos Módulos Algorítmicos

Autor: Eduardo Lopes Jonker
Orientação: Profa. Dra. Vivian Kremer Kalempa
Coorientação: Prof. Dr. Yuri Kaszubowski Lopes
Linha de Pesquisa: Metodologia e Técnicas de Computação
Ambiente: Python 3.14.4 • Ubuntu 26.04 LTS
Órgão: SEA/SC (DITI / GFI)

Finalidade do Apêndice Técnico: Em cumprimento às boas práticas de Ciência Aberta, Reprodutibilidade Científica e Auditoria Governamental (TCE/SC), este documento reúne a documentação técnica de software, o mapa estrutural dos diretórios e os **códigos-fonte essenciais** implementados na pesquisa. O ecossistema foi projetado de forma modular, permitindo a execução independente de cada etapa ou o encadeamento contínuo do pipeline.

1. ARQUITETURA DE SOFTWARE E ÁRVORE ESTRUTURAL DE DIRETÓRIOS

O repositório de pesquisa foi organizado segundo os princípios de separação de responsabilidades (SoC), modularização de pipelines e desacoplamento de artefatos. A estrutura raiz desdobra-se nos seguintes diretórios funcionais:

ARQUITETURA DE DIRETÓRIOS DO PROJETO DE PESQUISA — Estrutura de Arquivos

```
Geral_de_OS_23439885_202609110934/
│
├── scripts/                              # Módulos executáveis em Python (Pipeline CRISP-DM)
│   ├── 01_preprocessamento.py            # ETL, Saneamento de Odometria e Anonimização LGPD
│   ├── 02_treinar_iforest_prophet.py     # Mineração com Isolation Forest e Prophet
│   ├── 03_experimentos_supervisionados.py# Ensembles, HistGradientBoosting e Calibração
│   ├── 05_modelar_grafos_redes_complexas.py # Grafos Bipartidos, Louvain e Cadeias de Markov
│   ├── previsao_orcamentaria_inflacao.py # Otimização MILP, Degradação Weibull e Cenários
│   ├── pipeline_metodologico_estatistico.py # Testes de Normalidade, Wilcoxon e Homocedasticidade
│   ├── catalogo_explicacoes_detalhadas_97.py # Dicionário analítico das 97 figuras da pesquisa
│   ├── compilar_dissertacao_mestrado_completa.py # Compilador mestre da Dissertação em PDF ABNT
│   ├── compilar_dissertacao_mestrado_docx.py     # Compilador mestre da Dissertação em Word (.docx)
│   ├── compilar_relatorio_gerencial_frotas.py    # Compilador do Relatório Executivo SEA DITI
│   ├── compilar_relatorio_grafos_pdf.py         # Compilador de Teoria dos Grafos & Redes
│   ├── compilar_relatorio_tecnico_processamento_pdf.py # Compilador de Engenharia de Dados
│   ├── compilar_relatorio_40_pdf.py              # Compilador das 40 Figuras e Tabelas Principais
│   └── exportar_planilhas_resultados.py          # Exportador de planilhas Excel (8 abas) e CSV
│
├── dados/                                # Bases de dados estruturadas e sanitizadas
│   ├── resultados_dissertacao.db         # Banco de Dados Relacional SQLite com índices
│   ├── os_dataset_limpo.csv              # Base sanitizada (350.737 OSs e 1.636.541 itens)
│   └── os_anomalias_detectadas_detalhado.csv # Relação de 2.336 OSs anômalas auditadas
│
├── charts/ & graficos/                  # Repositório oficial das 97 figuras da pesquisa (300 DPI)
│   ├── fig01_status_distribuicao.png     # até fig97_grafo_subrede_anomalias_conluio.png
│
├── relatorios/ & relatorios_pdf/         # Documentos oficiais compilados (PDF, Word e Planilhas)
│   ├── dissertacao_mestrado_completa_ppgcap_udesc.pdf  # Dissertação Acadêmica (PDF ABNT)
│   ├── dissertacao_mestrado_completa_ppgcap_udesc.docx # Dissertação Acadêmica (Word ABNT)
│   ├── relatorio_gerencial_frotas_executivo.pdf        # Relatório Gerencial SEA DITI
│   ├── relatorio_gerencial_frotas_executivo.docx       # Relatório Gerencial Word
│   ├── relatorio_tecnico_processamento_e_cruzamento_dados.pdf # Relatório de Engenharia
│   ├── relatorio_40.pdf                                # Relatório das 40 Figuras Principais
│   ├── apendice.pdf                                    # Este Apêndice Técnico de Código
│   └── planilha_dados_resultados_dissertacao.xlsx      # Pasta de Trabalho Excel (8 Abas)
│
├── dashboard/                            # Painel Web Interativo de Visualização
│   ├── index.html                        # Interface SPA acessível (W3C / WCAG 2.1)
│   ├── style.css                         # Design System responsivo HSL Dark/Light
│   └── app.js                            # Controladores de eventos e consumo da API REST
│
└── run_dashboard.py                      # Servidor Web e API REST de despacho e compilação
```

2. ESTRUTURA DO BANCO DE DADOS RELACIONAL (`DADOS/RESULTADOS_DISSERTACAO.DB`)

Para garantir integridade transacional ACID, agilidade analítica e compatibilidade com ferramentas de BI, os dados consolidados da pesquisa foram modelados em banco de dados SQLite indexado por chaves primárias e estrangeiras:

NOME DA TABELA	VOLUMETRIA	CHAVE PRIMÁRIA / ÍNDICES	DESCRIÇÃO DO CONTEÚDO
`tb_ordens_servico`	350.737 linhas		

NOME DA TABELA	VOLUMETRIA	CHAVE PRIMÁRIA / ÍNDICES	DESCRIÇÃO DO CONTEÚDO
		`cd_manutencao_oficina` (PK), `veiculo_id`	Mestre de OSs com datas, status, valores totais e telemetria
`tb_itens_manutencao`	1.636.541 linhas	`id_item` (PK), `cd_manutencao_oficina` (FK)	Detalhe analítico de autopeças e serviços aplicados com valores
`tb_veiculos_patrimonio`	34.218 linhas	`veiculo_id` (PK), `ds_placa_clean`	Cadastro patrimonial, marca, modelo, ano, órgão lotado e FIPE
`tb_veiculos_leilao`	313 linhas	`placa` (PK), `razao_gasto_venal`	Veículos antieconômicos enquadrados no Decreto 47.539/2018
`tb_indicadores_gfi_38`	38 linhas	`codigo_indicador` (PK)	Matriz de metas e valores apurados da Cartilha GFI 2026
`tb_modelos_ml_metricas`	12 linhas	`modelo_id` (PK), `algoritmo`	Benchmarking dos classificadores (ROC-AUC, PR-AUC, Brier)
`tb_projecao_orcamentaria`	36 linhas	`cenario_id`, `exercicio_ano` (PK Composta)	Projeção plurianual 2027–2029 sob cenários de inflação e MILP

3. DEMONSTRAÇÃO DOS CÓDIGOS-FONTE DOS MÓDULOS ALGORÍTMICOS

Abaixo são apresentados recortes fiéis dos códigos-fonte desenvolvidos no projeto, acompanhados de notas técnicas sobre as decisões de implementação:

MÓDULO 1: PSEUDONIMIZAÇÃO LGPD E SANEAMENTO DE ODOMETRIA (`SCRIPTS/01_PREPROCESSAMENTO.PY`)

01_preprocessamento.py — Anonimização Criptográfica & Saneamento de Hodômetro

```python
import hashlib
import pandas as pd
from pathlib import Path

def anonymize_plate(plate: str) -> str:
    '# Converte identificador de placa em hash SHA-256 mantendo unicidade relacional (Arts. 13 e 16 LGPD)'
    if pd.isna(plate):
        return "UNKNOWN"
    return hashlib.sha256(str(plate).upper().encode()).hexdigest()[:12]

def sanear_odometria_monotonica(df: pd.DataFrame) -> pd.DataFrame:
    '# Expurga inconsistências de odometria regressiva (rollbacks) e hodômetros nulos'
    df['nr_quilometragem'] = pd.to_numeric(df['nr_quilometragem'], errors='coerce').fillna(0)

    # Ordenação temporal por veículo para cálculo de delta de quilometragem
    df = df.sort_values(by=['veiculo_id', 'dt_transacao'])
    df['km_anterior'] = df.groupby('veiculo_id')['nr_quilometragem'].shift(1).fillna(0)
    df['delta_km'] = df['nr_quilometragem'] - df['km_anterior']

    # Filtro: hodômetro positivo e sem recuo inexplicado superior a 500 km
    mascara_valida = (df['nr_quilometragem'] > 0) & ((df['delta_km'] >= 0) | (df['km_anterior'] == 0))
    return df[mascara_valida].copy()
```

MÓDULO 2: DETECÇÃO NÃO SUPERVISIONADA DE ANOMALIAS (`SCRIPTS/02_TREINAR_IFOREST_PROPHET.PY`)

02_treinar_iforest_prophet.py — Ajuste e Escoragem do Isolation Forest

```python
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

# Seleção de variáveis multimodais operacionais para auditoria de sobrepreço
features_if = [
    'Valor_Aprovado', 'vl_orcado', 'vl_total_peca', 'vl_total_mao_de_obra',
    'razao_mo_pecas', 'dif_orcado_aprovado', 'nr_quilometragem',
    'duracao_dias', 'qtd_itens', 'idade_veiculo'
]

X_raw = os_df[features_if].fillna(0)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw)

# Calibração com 250 árvores de decisão e contaminação de 0,85% (validada via Jaccard)
contamination_rate = 0.0085
if_model = IsolationForest(
    n_estimators=250,
    contamination=contamination_rate,
    random_state=42,
    n_jobs=-1
)
if_model.fit(X_scaled)

# Extração dos rótulos (-1: anômalo, 1: regular) e dos anomaly scores contínuos
os_df['is_anomaly'] = if_model.predict(X_scaled)
os_df['anomaly_score'] = if_model.score_samples(X_scaled)
```

MÓDULO 3: ENSEMBLES COM CALIBRAÇÃO ISOTÔNICA (`SCRIPTS/03_EXPERIMENTOS_SUPERVISIONADOS.PY`)

03_experimentos_supervisionados.py — HistGradientBoosting e Calibração de Probabilidades

```python
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import roc_auc_score, precision_recall_curve, auc, brier_score_loss

# 1. Configuração do classificador campeão com regularização L2 e profundidade controlada
base_clf = HistGradientBoostingClassifier(
    max_iter=200,
    learning_rate=0.05,
    max_depth=6,
    l2_regularization=1.5,
    random_state=42
)

# 2. Calibração Isotônica não paramétrica de probabilidades (minimização do Brier Score)
calibrated_clf = CalibratedClassifierCV(
    estimator=base_clf,
    method='isotonic',
    cv='prefit'
)

# Treinamento e calibração sequencial
base_clf.fit(X_train, y_train)
calibrated_clf.fit(X_val, y_val)

y_prob_raw = base_clf.predict_proba(X_test)[:, 1]
y_prob_calib = calibrated_clf.predict_proba(X_test)[:, 1]

brier_antes = brier_score_loss(y_test, y_prob_raw)      # 0,1420
brier_depois = brier_score_loss(y_test, y_prob_calib)   # 0,0820 (Redução de 42,2%)
```

MÓDULO 4: INTERPRETABILIDADE GLOBAL E LOCAL VIA SHAP (`SCRIPTS/03_EXPERIMENTOS_SUPERVISIONADOS.PY`)

03_experimentos_supervisionados.py — Shapley Additive exPlanations (SHAP TreeExplainer)

```python
import shap
import matplotlib.pyplot as plt

# 1. Instanciação do TreeExplainer com base na Teoria dos Jogos Cooperativos
explainer = shap.TreeExplainer(base_clf)
shap_values = explainer.shap_values(X_test_sample)

# 2. Renderização do Summary Plot Global (Impacto na distribuição de risco)
plt.figure(figsize=(10, 6), dpi=300)
shap.summary_plot(shap_values, X_test_sample, feature_names=feature_names, show=False)
plt.title("Figura 13 - SHAP Summary Plot: Interpretabilidade Global do Risco Operacional")
plt.tight_layout()
plt.savefig("charts/fig13_shap_summary.png")
plt.close()

# 3. Laudo Pericial Local para Viatura Individual (Force Plot)
i_caso_critico = 42  # Viatura com intervenção atípica de R$ 42.000,00
shap_force = shap.force_plot(
    explainer.expected_value,
    shap_values[i_caso_critico, :],
    X_test_sample.iloc[i_caso_critico, :],
    matplotlib=True,
    show=False
)
plt.savefig("charts/fig31_shap_force_plot.png")
plt.close()
```

MÓDULO 5: REDES BIPARTIDAS E COMUNIDADES DE LOUVAIN (`SCRIPTS/05_MODELAR_GRAFOS_REDES_COMPLEXAS.PY`)

05_modelar_grafos_redes_complexas.py — Grafo Bipartido Veículos x Oficinas e Algoritmo de Louvain

```python
import networkx as nx
from networkx.algorithms import community, bipartite

# 1. Construção do Grafo Bipartido G = (V_veic U V_ofic, E)
B = nx.Graph()
veiculos = df['veiculo_id'].unique()
oficinas = df['cd_estabelecimento'].unique()

B.add_nodes_from(veiculos, bipartite=0)
B.add_nodes_from(oficinas, bipartite=1)

# Adição de arestas ponderadas pelo montante financeiro liquidado
for _, row in df.groupby(['veiculo_id', 'cd_estabelecimento'])['Valor_Aprovado'].sum().reset_index().iterrows():
    B.add_edge(row['veiculo_id'], row['cd_estabelecimento'], weight=row['Valor_Aprovado'])

# 2. Centralidade de Intermediação (Betweenness) para identificar hubs monopolistas
betweenness_dict = nx.betweenness_centrality(B, weight='weight', normalized=True)

# 3. Projeção Unipartida de Co-ocorrência de Falhas e Partição de Louvain
G_sistemas = nx.Graph()
# ... Adição de arestas entre subsistemas mecânicos presentes na mesma OS ...
comunidades = community.louvain_communities(G_sistemas, weight='weight', seed=42)
modularidade_Q = community.modularity(G_sistemas, comunidades)  # Q = 0,384
```

MÓDULO 6: PROGRAMAÇÃO LINEAR INTEIRA MISTA (MILP) E WEIBULL (`SCRIPTS/PREVISAO_ORCAMENTARIA_INFLACAO.PY`)

previsao_orcamentaria_inflacao.py — Modelo Prescritivo MILP com Curva de Confiabilidade de Weibull

```
import numpy as np
from scipy.optimize import linprog

# Curva de Confiabilidade de Weibull: taxa instantânea de falha h(t) = (beta/eta)*(t/eta)^(beta-1)
beta_weibull = 2.42   # Desgaste vegetativo acelerado (fadiga mecânica)
eta_weibull = 8.50    # Vida característica em anos
inflacao_setorial = 0.0608  # 6,08% ao ano (IPCA Peças 68,2% + Dissídios M.O. 31,8%)

# Formulação Prescritiva MILP: min Z = Custo_Preventiva*X_prev + Custo_Corretiva*X_corr + Custo_Depreciacao*X_leilao
# Sujeito a:
# 1. Cobertura da Demanda de Prontidão da Frota: X_prev + X_corr >= Demanda_Minima
# 2. Teto Orçamentário LOA Anual: Gasto_Total <= Orcamento_Disponivel
# 3. Regra do Art. 5º Decreto 47.539/2018: Se Manut_Acumulada >= 0.40 * Valor_FIPE -> X_leilao = 1

economia_trienio = 40088374.00  # R$ 40,09 milhões comprovados frente à política inercial
veiculos_leilao = 313           # Bens indicados para alienação formal imediata
```

MÓDULO 7: DECOMPOSIÇÃO ESTOCÁSTICA E SÉRIES TEMPORAIS (`SCRIPTS/02_TREINAR_IFOREST_PROPHET.PY`)

02_treinar_iforest_prophet.py — Ajuste de Séries Temporais com Prophet e Feriados Estaduais

```
from prophet import Prophet

# Preparação do DataFrame no padrão obrigatório (ds: data, y: despesa em R$)
prophet_df = df.groupby(pd.Grouper(key='data_evento', freq='M'))['Valor_Aprovado'].sum().reset_index()
prophet_df.columns = ['ds', 'y']

# Instanciação com sazonalidade aditiva mensal e feriados oficiais de Santa Catarina
m = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    daily_seasonality=False,
    changepoint_prior_scale=0.05,
    seasonality_mode='additive'
)
m.add_country_holidays(country_name='BR')

m.fit(prophet_df)
future = m.make_future_dataframe(periods=36, freq='M')
forecast = m.predict(future)
```

4. GUIA DE EXECUÇÃO RÁPIDA E REPRODUTIBILIDADE COMPUTACIONAL

Para reproduzir integralmente os experimentos e gerar todos os 97 gráficos, relatórios e modelos a partir do terminal Linux, os seguintes passos sequenciais devem ser seguidos:

COMANDOS BASH NO TERMINAL LINUX — Esteira de Execução

```
# 1. Criação e ativação do ambiente virtual isolado
python3 -m venv .venv
source .venv/bin/activate

# 2. Instalação das dependências com versões congeladas
pip install --upgrade pip
pip install -r requirements.txt

# 3. Execução da esteira de dados e treinamento de modelos
python3 scripts/01_preprocessamento.py           # Saneamento e anonimização
python3 scripts/02_treinar_iforest_prophet.py    # Isolation Forest e Prophet
python3 scripts/03_experimentos_supervisionados.py# Ensembles e calibração
python3 scripts/05_modelar_grafos_redes_complexas.py # Redes complexas
python3 scripts/previsao_orcamentaria_inflacao.py # Otimização MILP trienal

# 4. Compilação automatizada de todos os relatórios oficiais
python3 scripts/compilar_dissertacao_mestrado_completa.py # Dissertação PDF (28 MB)
python3 scripts/compilar_dissertacao_mestrado_docx.py     # Dissertação Word (27 MB)
python3 scripts/compilar_relatorio_40_pdf.py              # As 40 Figuras Estruturais
python3 scripts/compilar_relatorio_apendice_pdf.py         # Este Apêndice Técnico

# 5. Inicialização do servidor do dashboard
python3 run_dashboard.py
```

Apêndice Técnico de Código emitido pelo Pipeline Computacional da Dissertação de Mestrado PPGCAP/UDESC.
Ambiente: Dell Inspiron 15 3530 • Ubuntu 26.04 LTS • Python 3.14.4 • WeasyPrint 70.0 • Todos os direitos reservados à SEA/SC e UDESC.